

Vision-Based Contact-Rich Robot Manipulation: A Scripted Oracle, Behaviour Cloning, and SAC on robosuite NutAssembly

Intelligent Control 2025–2026, Team 2 (robosuite / Franka Panda)

Abstract

We study vision-only control of a 7-DOF Franka Emika Panda for the robosuite `NutAssembly` task (round nut onto peg) under the benchmark constraint `use_object_obs=False`: the deployed policy consumes only the 84×84 `agentview` RGB image. Three controllers are compared along the project’s required axis (simple baseline vs. learning method): (i) a scripted *closed-loop oracle* that reads privileged simulator state and solves the task via a proportional finite-state machine ($\approx 83\%$ success, training-only upper bound); (ii) a frame-stacked, dual-head convolutional *behaviour-cloning* (BC) policy distilled from 24 successful oracle trajectories (3,084 image–action pairs), optionally refined with DAgger; and (iii) a *Soft Actor–Critic* (SAC) agent trained end-to-end from pixels with privileged reward shaping. We report the measured behaviour of each: the oracle is a reliable demonstrator and upper bound; the SAC agent learns a stable *reaching* behaviour ($\bar{r}_{ep} \approx 0.24\text{--}0.28$) but does not complete insertions within budget; and the vision BC policy reaches the handle to ~ 1 cm yet fails to grasp, yielding 0% task success. We analyse why — the sub-centimetre depth and wrist-yaw precision the grasp requires is not recoverable from 84×84 RGB — and document the design decisions (dual gripper head, frame stacking, action smoothing, CPU-budget encoder) made in response. The whole codebase, excists here, <https://github.com/Stavros-Stathopoulos/Vision-Based-Contact-Rich-Robot-Manipulation>

1 Methodology

Task and observation contract. The benchmark is robosuite’s `NutAssembly` with a `Panda` arm and `PandaGripper`, driven by the composite BASIC (operational-space) controller at `control_freq=20`. All scripts construct the environment through a single factory, `make_nut_env()`, which fixes the configuration: `single_object_mode=2` (the rotation-symmetric *round* nut, avoiding square-hole yaw alignment), a single `agentview` camera at 84×84 , `reward_shaping=True`, and crucially `use_object_obs=False` so that *no* privileged object coordinates ever enter a policy observation. The action vector is 7-dimensional: $a_{0:3}$ Cartesian end-effector velocity (each unit ≈ 0.05 m/step), $a_{3:6}$ orientation deltas (only the wrist-yaw a_5 is exercised), and a_6 the gripper command (-1 open, $+1$ close). A common `BaseController` interface (`reset(seed)`, `act(obs)→(action, info)`) makes the three controllers interchangeable and matches the graded submission contract; `info` is logging-only and never touches the environment, reward, termination, seed, or score.

(1) Scripted oracle (privileged, training-only). `OracleController` is a reactive expert and the demonstration generator. It never calls `act()`; instead `place_nut()` runs an 8-phase finite-state machine, reading live MuJoCo state each step: the nut *handle* site, nut *centre* body, peg body, and end-effector site. Each phase servos with a clipped proportional law

$$a_{0:3} = \text{clip}\left(\frac{K(p^* - p_{\text{eef}})}{0.05}, -1, 1\right), \quad a_5 = \text{clip}\left(\frac{2.5 \Delta\psi}{0.5}, -1, 1\right),$$

where p^* is the per-phase target and $\Delta\psi$ is the wrap-to- π error between the desired yaw and the measured finger-closing axis. Two geometric insights make it reliable: the gripper grasps the *protruding handle* (offset ~ 6 cm from the hole), so during transport the *nut centre* — not the end-effector — is servoed onto the peg; and the closing axis is aligned perpendicular to the handle rod so the fingers actually pinch it. The phases are: hover above handle (open) $\rightarrow$ descend $\rightarrow$ close (no XY push) $\rightarrow$ lift $\rightarrow$ transport nut-centre over peg $\rightarrow$ lower onto peg $\rightarrow$ release $\rightarrow$ retreat, gated by per-phase tolerances (e.g. XY error < 5 mm to stop hovering, place tolerance 4 mm). Success is the environment’s own `_check_success()`. The oracle is privileged and therefore *not* a valid vision submission; it serves as the upper-bound baseline and the expert.

(2) Vision behaviour cloning (the deliverable). The submitted policy is vision-only. The shared encoder `CNNEncoder` is a lightweight NatureCNN: three convolutions ($8 \times 8, s4 \rightarrow 4 \times 4, s2 \rightarrow 3 \times 3, s1$) mapping 84×84 through $20 \times 20 \rightarrow 9 \times 9 \rightarrow 7 \times 7$, flattened to $64 \cdot 7 \cdot 7 = 3136$ features and projected to a 256-d embedding. The aggressive stride-4 first layer is a deliberate compute/precision trade: it cuts the flattened dimension $\sim 6\times$ (vs. 18,496 for a stride-2 variant), which keeps training and inference tractable on CPU-only machines.

The trained behaviour-cloning network (verified from the saved `bc_model.pth` weights) is *frame-stacked* and *dual-headed*. Its input is $K=2$ consecutive frames concatenated on the channel axis ($6 \times 84 \times 84$; encoder first conv weight $32 \times 6 \times 8 \times 8$), giving the network a temporal/velocity cue that a single frame lacks. The shared 256-d embedding feeds two heads: a *continuous head* ($256 \rightarrow 128 \rightarrow 6$, `Tanh`) regressing $a_{0:6}$ under MSE, and a *separate binary gripper head* ($256 \rightarrow 128 \rightarrow 1$ logit) trained with binary cross-entropy against $\mathbf{1}[a_6 > 0]$. This split is the central design decision: a single MSE objective over the *bimodal* gripper channel averages toward “open” and never commands a firm close, so the policy never grasps; isolating the gripper as a classifier restores the close signal. Frame stacking is applied identically at train and inference time through one shared module (`frame_stack.py`: `StackedBCDataset` / `FrameStacker`), so the deployed input distribution matches training; stacking never crosses an episode boundary (padded by repeating the first frame). At deployment the controller /255-normalises pixels, feeds the `FrameStacker` buffer, runs the net, and hard-thresholds the gripper logit to ± 1 ; action smoothing is available as a benchmark-legal post-filter on the continuous deltas.

Dagger refinement. Because pure BC suffers compounding error on states the expert never visited, `train_dagger.py` rolls out the current policy but relabels *every visited state* with the oracle’s action, aggregates the (image, oracle action) pairs into the dataset, and retrains (3 iterations $\times$ 8 rollouts by default). The oracle relabels from true state and is therefore always correct, even on the policy’s off-distribution frames.

(3) SAC from pixels (learning baseline). `RobosuiteGymWrapper` adapts robosuite to the Gymnasium API: it extracts `agentview_image`, transposes HWC $\rightarrow$ CHW to a `uint8` (3,84,84) observation, and exposes the $[-1, 1]^7$ action box. SAC (`CnnPolicy`) reuses `CnnEncoder` as its `features_extractor`. To make the contact-rich task learnable, the wrapper adds a *dense reaching reward* $r_{\text{reach}} = \beta [1 - \tanh(\alpha d_{\text{eff}, \text{nut}})]$ with $\beta=0.5$, $\alpha=10$, on top of the environment’s shaped reward; like the oracle this reads privileged positions *only* for the reward, never as policy input, so the vision-only contract is preserved.

2 Experimental Procedure

Software/hardware. robosuite 1.5.2 on MuJoCo 3.3.0 (pinned: mujoco $\geq$ 3.10 breaks the OSC `mj_fullM` signature), PyTorch 2.2.1, Stable-Baselines3 2.3.0 (without `[extra]`), Gymnasium 0.29.1, Python 3.12 in a `.venv`. Offscreen rendering is required for camera observations; headless runs set `MUJOCO_GL=egl`. The device is auto-detected (CUDA else CPU); on CPU the thread count is capped (4) to keep the machine responsive.

Orchestration and the MuJoCo memory leak. `run_pipeline.py` runs each core stage — `wrapper` $\rightarrow$ `oracle` $\rightarrow$ `collect` $\rightarrow$ `train_bc` $\rightarrow$ `eval_bc`, with `train_rl/test_rl` opt-in (`-all`) and `train_dagger` invoked manually — as an *isolated subprocess*, skipping any stage whose output already exists (`-force` reruns). This structure is a direct response to a measured MuJoCo defect: RAM leaks across `env.reset()`, so a long single process is infeasible; per-stage isolation, forced `gc.collect()` per episode, and (for RL) `optimize_memory_usage=True` are the mitigations. Key knobs are environment variables (`DEMO_EPISODES`, `DAGGER_ITERS`, `BC_EPOCHS`, `SAC_TIMESTEPS`).

Demonstration collection. `collect_demos.py` runs the oracle until a target number of *successful* episodes is reached, recording the *pre-step* image paired with the action taken (`image_t` $\rightarrow$ a_t ; failures are discarded). The shipped dataset `expert_demos.npz` contains **3,084 frames from 24 successful episodes** (≈ 128 steps/episode), stored as `uint8` (3084,84,84,3) states, `float32` (3084,7) actions, and `episode_ids` 0...23 that mark episode boundaries so frame-stacking never bridges two episodes. Table 1 reports the per-dimension action statistics measured directly from the dataset; note $a_3, a_4 \equiv 0$ (orientation X/Y unused) and the strongly *bimodal* gripper channel.

BC / Dagger training. States are /255-normalised and reshaped to PyTorch CHW. Optimisation uses Adam ($\text{lr}=10^{-3}$, weight decay 10^{-4} in the Dagger retrainer), batch 32–64, with the composite loss $\mathcal{L} = \text{MSE}(\hat{a}_{0:6}, a_{0:6}) + \text{BCE}(\hat{g}, \mathbf{1}[a_6 > 0])$. Evaluation (`evaluate_bc.py`) rolls the policy deterministically and counts success via `_check_success()` — *not* reward >0 , which the shaped reward makes positive even on failure — at a long horizon (≥ 500 , the benchmark length, up to 900 to match the oracle).

SAC training. `train_rl.py` trains SAC with learning rate 3×10^{-4} , replay buffer 30k, batch 64, `optimize_memory_usage` (halving buffer RAM), at a shortened `horizon=200` to densify episodes. The shipped checkpoint is `SAC_100k.zip` (≈ 56 MB), i.e. 100k environment steps. Hyperparameters are summarised in Table 2.

Table 1: Per-dimension action statistics over the 3,084 demonstration frames (measured from `expert_demos.npz`). The gripper (a_6) is bimodal: 1,183 open vs. 1,901 close commands.

	Dim	Meaning	Mean	Std
	a_0	EEF x	+0.304	0.481
	a_1	EEF y	−0.124	0.407
	a_2	EEF z	−0.047	0.627
	a_3	orient. x	0.000	0.000
	a_4	orient. y	0.000	0.000
	a_5	wrist yaw	+0.061	0.423
	a_6	gripper	+0.233	0.973

Table 2: Key training settings (from the scripts and saved artifacts).

Encoder (shared)	
Conv stack	$8 \times 8 / s4$, $4 \times 4 / s2$, $3 \times 3 / s1$
Spatial	$84 \rightarrow 20 \rightarrow 9 \rightarrow 7$
Flatten / embed	$3136 \rightarrow 256$
Behaviour cloning	
Input	$K=2$ stacked frames ($6 \times 84 \times 84$)
Heads	cont. $256 \rightarrow 128 \rightarrow 6$ Tanh; grip $256 \rightarrow 128 \rightarrow 1$
Loss	MSE (cont) + BCE (gripper)
Optimiser	Adam 10^{-3} , wd 10^{-4} , batch 32–64
Dataset	3,084 frames / 24 oracle episodes
SAC	
Policy	CnnPolicy + CnnEncoder (256)
lr / buffer	3×10^{-4} / 30k, batch 64
Reward	shaped + $0.5[1 - \tanh(10 d_{\text{eeef,nut}})]$
Horizon / steps	200 / 100k (SAC_100k.zip)

3 Analysis

Why the oracle works. The oracle sidesteps perception entirely: operating on exact handle/centre/peg coordinates, the proportional FSM reduces the problem to sequential set-point regulation with generous tolerances. Its two non-trivial choices — pinching the offset *handle* and servoing the nut *centre* (not the end-effector) over the peg — are exactly the geometric facts a vision policy must *infer* but cannot measure. This is why it is a fair upper bound and an ideal relabelling expert, but disqualified as a submission.

Why the gripper head is dual. The demonstration gripper channel is bimodal (Table 1: 38% open / 62% close, std 0.97). Under a single MSE regression head, the minimiser of $\mathbb{E}[(\hat{g} - g)^2]$ is the *mean* ($\approx +0.23$), a soft value that, after thresholding, rarely commits to a firm close at the right instant — the policy hovers and never grasps, and since grasp gates lift/transport/insert, the whole task fails. Re-casting the gripper as a Bernoulli decision with a dedicated BCE head removes the averaging pathology. This is the single most important architectural lesson of the project and is why the deployed model carries two heads rather than one.

Why frame stacking and smoothing. A single 84×84 frame is nearly static between consecutive control steps; stacking $K=2$ frames supplies an implicit velocity signal (approach speed, contact onset) that a Markovian single-frame input cannot encode, at negligible cost (6 vs. 3 input channels). EMA action smoothing at deployment suppresses the high-frequency jitter that an open-loop regressor would otherwise inject into a contact-rich phase. Both are explicitly benchmark-legal techniques (frame stacking, action smoothing).

Why vision BC still reaches 0% — a measured failure, not a guess. The cloned policy executes the *coarse approach* correctly: the arm reaches within ~ 1 cm of the handle. It then fails to fire a reliable “close.” The gripper head fits the *training* frames essentially perfectly (saturated logits) yet does not generalise, because the discriminative

Table 3: Measured controller outcomes on the round-nut `NutAssembly` task. Success is `_check_success()`; the oracle uses privileged state and is training-only.

Controller	Observation	Outcome
Heuristic FSM	time-only	open-loop; no closed grasp
Oracle	privileged	$\approx 83\%$ success (ceiling)
SAC (100k)	pixels	reach only, $\bar{r}_{\text{ep}} \approx 0.24\text{--}0.28$
BC + DAgger	pixels	~ 1 cm approach; 0% success

cue — sub-centimetre depth to the handle and the wrist-yaw that aligns the fingers to a ~ 6 cm offset rod — simply is not recoverable from a single 84×84 RGB view. The oracle reads these from `sim`; the camera cannot resolve them. DAgger cannot rescue this regime: when *every* rollout fails identically at the grasp, the aggregated states are all more *approach* frames, so relabelling adds no new information about the missing grasp decision. The benchmark-legal levers actually applied — more demonstrations, dropout/weight-decay regularisation, the dual head, frame stacking, action smoothing — improve the approach and the training fit but do not manufacture the missing pixels. Crucially, the obvious fixes (higher resolution, a wrist `eye_in_hand` camera) are *prohibited at scoring*: the fixed benchmark sensing is `agentview` at 84×84 , so they are valid only as training-time ablations, never as a runtime dependency of the submission.

Why SAC plateaus at reaching. The dense $1 - \tanh(10d)$ term is informative for *approach* but provides almost no gradient for the precise grasp-and-insert, which requires a long, low-probability sequence of correct contacts. Within 100k steps at horizon 200 the agent reliably optimises the easy component — driving the gripper to the nut — and its episodic return stabilises around 0.24–0.28, consistent with “reach but do not insert.” This mirrors the BC outcome from the opposite direction: both methods solve the visually-easy reaching sub-task and stall at the perception-limited grasp.

4 Results

Table 3 consolidates the measured outcomes. The privileged oracle is the only controller that completes the full pick–transport–insert sequence, at $\approx 83\%$ over repeated episodes; it is the demonstrator and the practical ceiling. Both vision-only learners solve the reaching sub-task but not the contact grasp: SAC stabilises at $\bar{r}_{\text{ep}} \approx 0.24\text{--}0.28$ (reach, no insertion), and the BC/Dagger policy — despite a near-perfect training fit and ~ 1 cm approach accuracy — scores 0% task success because the grasp-critical depth and yaw are unobservable at 84×84 . The deliverables produced are the 3,084-frame demonstration set (`expert_demos.npz`), the dual-head BC weights (`bc_model.pth`, ≈ 3.8 MB), the 100k-step SAC checkpoint (`SAC_100k.zip`, ≈ 56 MB), and an oracle demonstration video (`oracle_demo.mp4`).

Reproducibility. Every number above is regenerable from the committed pipeline: `python -m src.models.test_oracle` for the oracle rate, `python -m src.models.evaluate_bc` (horizon 500) for BC success, and the SAC return from the Stable-Baselines3 Monitor log during `train_rl`. The single environment factory guarantees all three controllers face identical task configuration.

Takeaways and outlook. The project satisfies the required two-approach comparison (scripted baseline/oracle vs. BC/Dagger and SAC) and yields a clear, measured diagnosis rather than an unexplained 0%: at the fixed 84×84 `agentview` sensing, the bottleneck is *perceptual resolution at the grasp*, not optimisation or interface. The benchmark-legal levers that remain genuinely open are (i) training-time domain randomisation of nut/peg pose, lighting, and viewpoint to harden the approach and add recovery behaviour, the dimension the hidden stress tests reward, and (ii) richer temporal/recurrent state to better exploit the two-frame motion cue — both consuming only benchmark observations at deployment.